



МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего
образования

Московский авиационный институт
(Национальный исследовательский университет)

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806

Отчёт по лабораторной работе №1

Дисциплина: «Операционные системы и архитектура компьютера»

Тема: «Анализ профиля системных вызовов программы `ffmpeg`»

Выполнил:
студент группы М8О-206БВ-25
Ширанков М.С.

Проверил:
Ассистент Центра ТОП-ИТ
Ефимов Д.А.

Содержание

1	Цель работы	2
2	Ход работы	2
3	Получение полного профиля вызовов	2
3.1	Значение столбцов	3
3.2	Анализ результата	3
4	Разделение профиля по логическим категориям	3
4.1	Категория: память (%memory)	4
4.2	Категория: процессы и потоки (%process)	4
4.3	Категория: сигналы (%signal)	5
4.4	Категория: файловый ввод-вывод (%desc)	5
4.5	Категория: сеть (%network)	5
5	Итог работы	5

1 Цель работы

Построить профиль системных вызовов программы (`ffmpeg`) с помощью утилиты `strace`, получить статистику вызовов и разбить её по логическим подсистемам ядра (память, процессы/поток, файловый ввод-вывод, сигналы, сеть), не смешивая их между собой.

2 Ход работы

В качестве исследуемой команды используется перекодирование видео `zazu_input.mp4` с явным указанием видеокodeка `libx264`:

```
ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

Команда задействует сразу несколько подсистем ОС:

- чтение исходного видео и запись результирующего файла (файловый I/O);
- динамическую загрузку разделяемых библиотек (`libavcodec`, `libx264` и др.) при старте процесса;
- многопоточное кодирование кадров библиотекой `libx264` (создание потоков и их синхронизацию);
- управление памятью под буферы кадров и рабочие структуры codeка.

3 Получение полного профиля вызовов

Флаг `-c` переводит `strace` в режим сбора статистики: вместо построчной трассировки каждого вызова выводится статистическая таблица (число вызовов, суммарное и среднее время, количество вызовов, завершившихся ошибкой). Флаг `-f` обязателен для `ffmpeg`, так как codeк `libx264` создаёт дочерние потоки через `clone3()` - без `-f strace` видит только главный поток и теряет всю статистику по кодированию.

```
strace -f -c ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

Результат выполнения (перекодирование тестового видео `zazu_input.mp4` на 4-ядерной машине):

```
% time seconds usecs/call calls errors syscall
-----
98.19 7.442150 1842 4039 330 futex
0.63 0.047605 44 1063 mmap
0.29 0.021859 51 421 mprotect
0.27 0.020194 196 103 munmap
0.12 0.009123 37 242 read
0.12 0.008736 38 228 5 openat
0.09 0.006535 28 229 fstat
0.08 0.005932 26 223 close
0.06 0.004730 32 147 write
0.04 0.003178 167 19 clone3
0.03 0.002501 32 77 rt_sigprocmask
0.03 0.002185 66 33 brk
0.01 0.000900 45 20 rseq
0.01 0.000861 45 19 madvise
```

```

0.01 0.000823 41 20 set_robust_list
0.01 0.000426 30 14 4 prctl
0.00 0.000195 27 7 lseek
0.00 0.000163 27 6 pselect6
0.00 0.000155 31 5 sched_getaffinity
0.00 0.000129 64 2 2 ioctl
0.00 0.000116 19 6 rt_sigaction
0.00 0.000116 38 3 3 access
...

```

```

-----
100.00 7.579191 1090 6948 348 total

```

3.1 Значение столбцов

- **% time** - доля суммарного времени выполнения системных вызовов, пришедшаяся на данный вызов (не доля времени всей программы - I/O-ожидание и вычисления в пространстве пользователя сюда не входят);
- **seconds** - суммарное время, потраченное ядром на обработку всех вызовов этого типа;
- **usecs/call** - среднее время одного вызова в микросекундах;
- **calls** - сколько раз вызов был выполнен (суммарно по всем потокам, так как использован `-f`);
- **errors** - сколько из этих вызовов завершились с кодом ошибки (`-1` и установленным `errno`);
- **syscall** - имя системного вызова.

3.2 Анализ результата

Основная доля времени, проведённого в системных вызовах, приходится на `futex`. `Libx264` распараллеливает кодирование кадров/срезков (`slice-based threading`) между рабочими потоками по числу ядер CPU (здесь - 4), а сами потоки создаются вызовом `clone3`. Синхронизация между ними (очередь заданий, условные переменные, мьютексы в реализации `NPTL/glibc`) реализована поверх `futex`, поэтому именно он доминирует и по числу вызовов, и по суммарному времени. Остальные вызовы (`mmap`, `mprotect`, `munmap`, `brk`, `madvise`) - это управление памятью под буферы кадров и стеки потоков, а `read/openat/fstat/close/write` - чтение исходного файла, загрузка динамических библиотек кодеков при старте и запись результата.

4 Разделение профиля по логическим категориям

Готовые классы `strace` (`%memory`, `%process`, `%signal`, `%network`, `%desc`, `%file`, `%ipc`, `%clock`) позволяют собрать статистику только по вызовам одной подсистемы, не смешивая их с остальными. Для каждой категории трассировка запускалась отдельным прогоном, чтобы категории физически не пересекались в выводе:

```
# memory: mmap/munmap/mprotect/brk/madvise
strace -f -c -e trace=%memory -o mem_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# processes/threads: clone3/execve/wait4/exit_group
strace -f -c -e trace=%process -o process_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# signals: rt_sigaction/rt_sigprocmask/tgkill
strace -f -c -e trace=%signal -o signal_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# file descriptors / I/O: read/write/open/close/fstat...
strace -f -c -e trace=%desc -o io_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4

# network: socket/connect/send*/recv*
strace -f -c -e trace=%network -o net_profile.log \
    ffmpeg -i zazu_input.mp4 -c:v libx264 zazu_output.mp4
```

4.1 Категория: память (%memory)

```
% time seconds usecs/call calls errors syscall
-----
51.55 0.046484 43 1063 mmap
23.83 0.021487 51 421 mprotect
20.51 0.018489 179 103 munmap
 3.16 0.002851 150 19 madvise
 0.86 0.000774 23 33 brk
 0.09 0.000079 79 1 mlock
-----
100.00 0.090164 54 1640 total
```

Преобладают `mmap/mprotect/munmap` - это загрузка динамических библиотек кодеков (`ld.so` отображает каждую `.so` в память и выставляет права страниц) и выделение буферов под кадры или потоковые стеки.

4.2 Категория: процессы и потоки (%process)

```
% time seconds usecs/call calls errors syscall
-----
82.63 0.002560 134 19 clone3
17.37 0.000538 538 1 execve
-----
100.00 0.003098 154 20 total
```

Один `execve` - запуск самого процесса `ffmpeg`; 19 вызовов `clone3` - создание потоков пула кодирования `libx264`. При просмотре построчного лога с `-f` по такому количеству потоков часто встречаются строки вида `<unfinished ...>` и `<... resumed>`. `Strace` показывает чередование: если, пока один поток ожидает завершения системного вызова (например, блокирующего `futex` или `read`), планировщик переключается на другой поток и тот тоже

входит в `syscall`, `strace` не может напечатать оба вызова как непрерывные строки - начатый, но ещё не завершившийся вызов помечается `<unfinished ...>`, а когда управление возвращается к его поток и вызов действительно завершается, соответствующая строка получает пометку `<... resumed>` с итоговым результатом.

4.3 Категория: сигналы (%signal)

```
% time seconds usecs/call calls errors syscall
-----
93.84 0.002454 31 77 rt_sigprocmask
 6.16 0.000161 26 6 rt_sigaction
-----
100.00 0.002615 31 83 total
```

`rt_sigprocmask` вызывается при каждом создании потока (`glibc` маскирует сигналы на время инициализации нового потока), `rt_sigaction` - установка обработчиков (например, `SIGINT` для корректного завершения по `Ctrl+C`).

4.4 Категория: файловый ввод-вывод (%desc)

```
% time seconds usecs/call calls errors syscall
-----
57.66 0.046460 43 1063 mmap
10.93 0.008807 38 228 5 openat
 9.92 0.007994 32 243 read
 8.11 0.006533 28 229 fstat
 7.93 0.006390 28 223 close
 4.39 0.003540 24 147 write
 0.39 0.000312 44 7 pselect6
 0.26 0.000209 29 7 lseek
-----
100.00 0.080570 37 2158 9 total
```

Здесь видно чтение исходного `zazu_input.mp4`, запись результата и множество пар `openat/fstat/close` - это динамический загрузчик ищет и открывает разделяемые библиотеки кодеков при старте процесса (5 из 228 `openat` завершились ошибкой `ENOENT` - обычный перебор путей поиска `.so`-файлов).

4.5 Категория: сеть (%network)

Вывод пуст. Поскольку и вход, и выход - локальные файлы, `ffmpeg` в этом сценарии ни разу не обращается к сетевому стеку (ни `socket`, ни `connect`, ни `sendto/recvfrom`).

5 Итог работы

В результате выполнения лабораторной работы были получены практические навыки профилирования реальной многопоточной программы: сбор статистики (`-с`), разделение по классам системных вызовов (`-e trace=%class`), а также разбор сырых трасс для глубокого изучения понимания работы программы.